Numpy-Accelerated Partition Infrastructure

Summary of `partition_numpy.py`

(a) Theory

Partition representation

A partition of L into k parts is an unordered multiset of positive integers summing to L. The natural representation for algorithmic work is the **multiplicity encoding**: store the distinct part values $v_1 < v_2 < \cdots < v_{n_d}$ and their multiplicities $m_1, m_2, \ldots, m_{n_d}$, with $\sum_i v_i m_i = L$ and $\sum_i m_i = k$. For a typical partition of L, the number of distinct values is $n_d = O(\sqrt{L})$, so all operations on this representation run in $O(\sqrt{L})$ time and memory rather than $O(k) = O(\sqrt{L}\log L)$ for the flat list.

The three move types

The Metropolis-Hastings samplers in `mc_fast.py` and `mc_uniform_fast.py` explore the space of partitions via three elementary moves:

- **Transfer**: $(a, b) \mapsto (a+1, b-1)$, valid when $a \in \text{vals}$, $b \in \text{vals}$, $b \geq 2$, $a+1 \neq b$, and (if $a = b$) $m_a \geq 2$. Preserves both k and L.
- **Split**: $a \mapsto (c,\ a-c)$ for $1 \leq c \leq \lfloor a/2 \rfloor$, valid when $a \geq 2$. Increases k by 1.
- **Merge**: $(c, d) \mapsto c + d$ for $c \leq d$ both present (with $m_c \geq 2$ when $c = d$). Decreases k by 1.

Together these three types make the Markov chain irreducible on the full set of partitions of L: transfers explore within a fixed part count, while splits and merges connect partitions of different part counts.

Move count formulas

The Metropolis-Hastings acceptance ratio requires the number of valid moves of the chosen type from both the current and proposed states. Counting by direct enumeration would cost $O(n_d^2)$ for transfers and merges; the formulas below reduce this to $O(n_d)$:

$$N_{\text{transfer}} = n_v \cdot n_{\text{dec}} - n_{\text{adj}} - n_{\text{ei}}$$

where $n_v = n_d$ (all values are incrementable), $n_{\text{dec}} = \#\{v : v \geq 2\}$ (decrementable values), $n_{\text{adj}} = \#\{v : v+1 \in \text{vals}\}$ (adjacent pairs that would be null moves), and $n_{\text{ei}} = \#\{v \geq 2 : m_v = 1\}$ (decrementable values with multiplicity 1, for which the same-value transfer $v \to v$ is invalid).

$$N_{\text{split}} = \sum_{v \in \text{vals}} \lfloor v/2 \rfloor$$

$$N_{\text{merge}} = \binom{n_d}{2} + \#\{i : m_i \geq 2\}$$

Weighted measure

`mc_fast.py` samples under a weighted measure $\tilde{w}(\lambda) \propto k!\, 2^k / \prod_j m_j!$, whose log-weight is

$$\log \tilde{w}(\lambda) = \log \Gamma(k+1) + k \log 2 - \sum_j \log \Gamma(m_j + 1).$$

This is evaluated via `lgamma` to avoid overflow in $k!$ at large k.

(b) What the script does

`partition_numpy.py` is a shared library, not a standalone script. It provides all partition data structures and operations used by `mc_fast.py` and `mc_uniform_fast.py`, and is imported by `rld_fast.py` indirectly via `mc_fast`. It contains no `__main__` block and produces no output when run directly.

The module provides six groups of functions:

1. **Construction helpers**: convert between flat lists, multiplicity arrays, and canonical tuples; initialise a near-uniform starting partition.
2. **Insert/remove primitives**: $O(n_d)$ operations to increment or decrement the multiplicity of a single value, handling insertion and deletion of entries.
3. **Move applicators**: apply a transfer, split, or merge to a state, returning the new `(vals, mults, k)` triple.
4. **Move count functions**: compute N_{transfer}, N_{split}, N_{merge} in $O(n_d)$ without enumerating moves.
5. **Move enumerators**: return the full list of valid moves of each type, used when uniform sampling within a move type is required.
6. **Autocorrelation utilities**: vectorised autocorrelation function and integrated autocorrelation time estimator, shared by both samplers.

(c) How the script does it

State representation

`vals` and `mults` are `int32` numpy arrays of length n_d, always maintained in ascending order of part value. The invariants $\sum_i \text{vals}[i] \cdot \text{mults}[i] = L$ and $\sum_i \text{mults}[i] = k$ are preserved by every move applicator.

Insert/remove primitives: `_inc` and `_dec`

Both use `np.searchsorted` for $O(\log n_d)$ lookup and `np.insert` / `np.delete` for $O(n_d)$ array manipulation, keeping the arrays sorted at all times. `_inc(vals, mults, v)` increments the multiplicity of v, inserting a new entry if v is not present. `_dec(vals, mults, v)` decrements the multiplicity of v, removing the entry if it reaches zero. Both return new arrays without modifying the inputs.

Move applicators

Each applicator is composed directly from `_inc` and `_dec`:

- `apply_transfer_np`: dec a, inc $a+1$, dec b, inc $b-1$ (four calls).
- `apply_split_np`: dec a, inc c, inc $a-c$ (three calls, $k+1$).
- `apply_merge_np`: dec c, dec d, inc $c+d$ (three calls, $k-1$).

Move count functions

`n_transfers_np` applies the formula $n_v \cdot n_{\text{dec}} - n_{\text{adj}} - n_{\text{ei}}$ using numpy masking for the decrementable subset and a Python set lookup for the adjacency count. `n_splits_np` is a single vectorised `(vals // 2).sum()`. `n_merges_np` computes the combinatorial term $n_d(n_d-1)/2$ plus the count of high-multiplicity entries.

Move enumerators

`all_transfers_np` iterates over all (a, b) pairs in $O(n_d^2)$, filtering by the validity conditions. `all_splits_np` iterates over all (a, c) pairs in $O(\sum v)$. `all_merges_np` uses a set to collect distinct unordered pairs in $O(n_d^2)$. These are used in the uniform sampler (`mc_uniform_fast.py`), where uniform sampling within a move type requires the full move list; the weighted sampler (`mc_fast.py`) instead samples moves without enumeration.

Initialisation

`init_arrays(L)` constructs a near-uniform starting partition with $k_0 = \max(1, \lfloor 1.28\sqrt{L} \rfloor)$ parts, distributing L as evenly as possible (parts of size $\lfloor L/k_0 \rfloor$ and $\lfloor L/k_0 \rfloor + 1$). This initialises close to the grand canonical mode, which is well below the canonical ($\sqrt{L}\log L$) typical part count; the equilibration period of the sampler allows the chain to reach the correct stationary distribution.

Autocorrelation utilities

`autocorrelation_np(series, max_lag)` computes the normalised autocorrelation function using direct numpy dot products, truncating at $\text{max\_lag} = \min(500, n/4)$. It mean-centres the series before computing, and returns 1.0 at lag 0 by construction.

`tau_int_np(acf)` implements the standard self-consistent truncation rule: accumulate $\tau = 0.5 + \sum_{t=1}^{T} \text{acf}[t]$, stopping when $T \geq 5\tau$. This is the Madras--Sokal window estimator used throughout the MCMC literature.

(d) Output produced

None. `partition_numpy.py` is a library module with no standalone output.

(e) What we learn

The multiplicity encoding gives a 10–50× speedup over flat lists. For a typical partition of $L = 500$, $n_d \approx 30$ while $k \approx 60$; all move operations cost $O(30)$ rather than $O(60)$. At $L = 2000$, $n_d \approx 60$ while $k \approx 145$, and the advantage grows. The speedup is essential for the large-L runs in `rld_fast.py` ($L = 1000$, 3×10^6 steps) and `mc_uniform_fast.py` ($L = 10000$).

The $O(n_d)$ move count formulas avoid the $O(n_d^2)$ cost of full enumeration. The Metropolis--Hastings correction factor $N_{\text{fwd}}/N_{\text{rev}}$ must be recomputed at every step; doing so by full enumeration would dominate the per-step cost at large n_d. The closed-form formulas for N_{transfer}, N_{split}, and N_{merge} reduce this to a small number of numpy operations regardless of n_d.

Move enumerators and count functions serve different roles. The uniform sampler (`mc_uniform_fast.py`) needs the full move list to draw a move uniformly at random within a type; the

weighted sampler (`mc_fast.py`) can sample a move without enumeration and uses only the count functions for the acceptance ratio. The module provides both to support both use cases without redundancy.